```
import numpy as np
import matplotlib.pyplot as plt
# Define function
def function(x):
    return (x + 3) ** 2

# Define gradient (derivative)
def gradient(x):
    return 2 * (x + 3)

# Initial values
x = 2
learning_rate = 0.1 # Controls the step size
iterations = 20

x_values = []
#( Creates an empty list named x_values.It stores the x values of each iteration. We use these values later to plot red dots on the graph.)
print("Starting point: x =", x)

# Gradient Descent
for i in range(iterations):

    x_values.append(x)

    # Update x
    x = x - learning_rate * gradient(x)

    print("Iteration", i + 1,
          ": x =", round(x, 6),
          "y =", round(function(x), 6))
print("Theoretical Minimum: x = -3, y = 0")
```

```
# Final result
print("\nLocal Minimum:")
print("x =", round(x, 6))
print("y =", round(function(x), 6))

# Plot function
x_plot = np.linspace(-6, 3, 100)
y_plot = function(x_plot)

plt.plot(x_plot, y_plot, label="y = (x + 3)^2")

plt.scatter(
  x_values,
  [function(i) for i in x_values],
  color="red",
  label="Gradient Descent"
)

plt.xlabel("x")
plt.ylabel("y")
plt.title("Gradient Descent to Find Local Minimum")
plt.legend()
plt.grid()

plt.show()
```

Gradient Descent is an optimization algorithm used to find the minimum value of a function by repeatedly updating the variable in the opposite direction of its gradient. In simple words, it helps a machine learning model find the best values of its parameters by gradually reducing its error. **Why do we use Gradient Descent?**

Suppose we train a machine learning model to predict students' marks.

- The model predicts marks.
- We compare predicted marks with actual marks.
- The difference is called error or loss.

- We want to reduce this error as much as possible.

Gradient Descent helps us reduce this error step by step until we reach a minimum.

Find the local minimum of the function:

y=(x+3)^2 ,Starting point:x=2,Learning rate:alpha=0.1 ,Number of iterations = 20.

Our goal is to find the value of x where y becomes minimum.

Step 1: Understand the functiony=(x+3)^2

This is a quadratic function. Its graph is U-shaped (parabola).

The minimum occurs at: x=-3, y=0 This is our theoretical minimum.

Step 2: Calculate the derivative (gradient)

First, we differentiate the function. y=(x+3)^2

{dy}/{dx}=2(x+3)

The gradient tells us the direction and steepness of the function.

Step 3: Apply the Gradient Descent formula

x_{new}=x_{old}- alpha*gradient

Where:

- \(x_{old}\) = Current value of x.
- \(\alpha\) = Learning rate (0.1).
- Gradient = Derivative of the function. Notice two things:
- The x value is moving towards -3.The y value is continuously decreasing towards 0.This means Gradient Descent is successfully finding the minimum.

Your graph contains two important things:

- Blue curve: Represents the function \(y=(x+3)^2\).
- Red dots: Represent the positions reached by Gradient Descent in each iteration.

How to understand it:

1. Initially, the algorithm starts at x = 2.
2. The red dots move downwards along the blue curve.
3. As iterations increase, the red dots approach the bottom of the curve.
4. The bottom of the curve represents the minimum point.
5. Why are the values different?
6. Because we performed only 20 iterations. The algorithm has approached the minimum but has not reached it exactly.
7. If we increase the number of iterations, x will get closer to -3 and y will get closer to 0.
8. Conclusion
9. The Gradient Descent algorithm successfully converged towards the minimum of the given function.

What is np.linspace()?Generates equally spaced numbers. np represents NumPy.

linspace() generates equally spaced numbers.-6 is the starting value.3 is the ending value.100 means it generates 100 equally spaced values.These values are used to draw a smooth curve.

What is plt.plot()?Draws a line graph.

What is plt.scatter()?Draws individual points. Or plt.legend: Displays the labels of the blue curve and red dots.

There are three types of Gradient Descent:

1. Batch Gradient Descent (BGD)
2. Stochastic Gradient Descent (SGD)
3. Mini-batch Gradient Descent (MBGD)

Comparison table

Parameter	**Batch GD**	**Stochastic GD**	**Mini-batch GD**
Data used	Entire dataset	One data point	Small group of data points
Updates	After processing all data	After each data point	After each mini-batch
Speed	Slower for large datasets	Faster updates	Generally efficient
Accuracy	Stable updates	Noisy updates	More balanced
Memory	High	Low	Medium
Convergence	Smooth	Fluctuating	Relatively stable
Computation	More per update	Less per update	Moderate per update

Explanation with example

Suppose we have a dataset containing 1000 training records.

- Batch Gradient Descent: Uses all 1000 records to calculate the error and update parameters once.
- Stochastic Gradient Descent: Uses one record at a time and updates parameters after every record.
- Mini-batch Gradient Descent: Uses a small group, such as 100 records, and updates parameters after processing each group.

In linear regression we use gradient descent The algorithm repeats these steps:

1. Predict the output.
2. Calculate the error.
3. Calculate gradients.
4. Update slope and intercept.
5. Repeat until the error becomes sufficiently small or a stopping condition is met.

No, Gradient Descent does not always converge to an optimum.

Its convergence depends on factors such as the learning rate, type of loss function, and starting point.

Reasons why Gradient Descent may not converge

1. High learning rate

If the learning rate is too high, the algorithm may take very large steps.It may overshoot the minimum and fail to converge.

2. Low learning rate:If the learning rate is too low, the algorithm takes very small steps.It may take a very long time to reach the minimum.

3. Local minima

In non-convex functions, there may be multiple local minima.Gradient Descent may get stuck in a local minimum instead of reaching the global minimum.

4. Saddle points

Sometimes the gradient becomes zero or nearly zero at a saddle point.The algorithm may slow down or stop progressing.

5. Poor initialization

- Choosing an unsuitable starting point can affect the optimization process, especially for non-convex functions.

How can we improve convergence?

1. Choose an appropriate learning rate.
2. Use a suitable number of iterations.
3. Normalize or standardize input data when appropriate.
4. Use optimization techniques such as Momentum or Adam.
5. Use a suitable initialization method.

Conclusion: Gradient Descent does not always guarantee convergence to the global optimum. However, with a suitable learning rate, proper initialization, and an appropriate loss function, it can converge successfully.